Zen Synthetic Data: Scalable Training Data Generation

Technical Report v2025.08

Zach Kelling
Zen LM Research Team
research@zenlm.org

August 2025

Abstract

We present the Zen Synthetic Data framework, a scalable pipeline for generating high-quality training data for instruction following, reasoning, domain specialization, and alignment. Zen Synthetic Data introduces constitutional synthetic generation (CSG), which applies a hierarchy of quality constraints during generation, and an automated quality scoring pipeline that filters generated data to match or exceed human-curated quality. A self-play data flywheel further iteratively improves data quality using the improving model. We use the Qwen3-based Zen models (dense 0.6B/4B/8B/32B and the Qwen3-30B-A3B mixture-of-experts variant, all Apache-2.0) as the generators and students in this pipeline. On downstream evaluations, models trained on synthetic CSG examples improve on MT-Bench and MMLU relative to training on equal volumes of web-scraped instruction data, and human raters consistently prefer CSG data over filtered web data.

1 Introduction

High-quality training data is the primary bottleneck for state-of-the-art language model development. Human annotation is slow, expensive, and difficult to scale. Web-scraped instruction data is abundant but noisy, biased toward certain domains and styles, and frequently low-quality.

Synthetic data generation offers a path to scalable, controllable, high-quality training data. However, naive synthetic generation fails: models trained on their own outputs degrade (model collapse), and unconstrained generation produces repetitive, biased, or inaccurate content.

The Zen Synthetic Data framework addresses synthetic generation at three levels:

1. **Constitutional generation**: Constraints applied during generation to ensure factual accuracy, diversity, and alignment.
2. **Automated quality scoring**: Multi-dimensional scoring pipeline that filters generated data to human-curator equivalence.
3. **Self-play flywheel**: Iterative improvement cycle where better models generate better data, which trains better models.

2 Constitutional Synthetic Generation

2.1 Constitutional Constraints

Constitutional AI for data generation extends the concept from alignment to training data quality. Each generated instruction-response pair is required to satisfy a hierarchy of constraints:

Level 1: Factual accuracy. Verifiable factual claims must be correct. Enforced by a fact-checking model and web retrieval verification.

Level 2: Instruction adherence. The response must fully address all components of the instruction. Enforced by an instruction-following evaluator.

Level 3: Diversity. Generated examples must be semantically distinct from existing training examples. Enforced by embedding-based deduplication with threshold sim < 0.85.

Level 4: Alignment. Responses must be helpful, harmless, and honest. Enforced by a reward model.

Level 5: Domain accuracy. For domain-specific data (medical, legal, code), specialized validators check domain-specific accuracy requirements.

2.2 Generation Pipeline

1. **Seed sampling**: Sample a diverse seed topic from a topic taxonomy covering 2,400 domains and subdisciplines.
2. **Instruction generation**: Generate a novel instruction for the topic using a Qwen3-based Zen model with diversity-promoting prompting.
3. **Response generation**: Generate a candidate response with chain-of-thought reasoning.
4. **Constitutional review**: The generating model self-critiques the response against each constitutional constraint.
5. **Revision**: If constraints are violated, the model revises the response.
6. **Quality scoring**: Multi-model scoring pipeline assigns quality scores (Section 3).
7. **Acceptance**: Examples with total quality score $> \tau = 0.78$ are accepted into the training corpus.

Table 1: Constitutional generation acceptance rates by constraint level

Constraint	Pass Rate (pre-revision)	Pass Rate (post-revision)
Factual accuracy	84.2%	94.8%
Instruction adherence	88.4%	97.2%
Diversity	91.2%	91.2%
Alignment	92.8%	98.4%
Domain accuracy (domain data)	78.4%	91.2%
Overall acceptance	62.4%	**84.8%**

3 Automated Quality Scoring

3.1 Scoring Dimensions

Each generated example is scored by a panel of specialized models on five dimensions:

Table 2: Quality scoring dimensions and models

Dimension	Scoring Model	Weight	Range
Factual accuracy	Fact-check classifier	0.25	0–1
Instruction following	Evaluator model	0.25	0–1
Response quality	Reward model	0.20	0–1
Diversity	Embedding distance	0.15	0–1
Formatting	Rule-based	0.15	0–1
Total score	Weighted average	1.00	0–1

3.2 Score Calibration

Scoring models are calibrated against human annotation on 10,000 examples. Calibration targets:

$$\hat{s}_{\text{model}}(x) \approx \mathbb{E}[\text{human rating}(x)] \tag{1}$$

Calibration accuracy (Pearson correlation with human ratings) per dimension:

Table 3: Score calibration against human raters

Dimension	Pearson r	RMSE
Factual accuracy	0.884	0.082
Instruction following	0.912	0.071
Response quality	0.876	0.094
Diversity	0.941	0.048
Formatting	0.968	0.031
Total score	0.924	0.062

4 Self-Play Data Flywheel

4.1 Architecture

The self-play flywheel operates across multiple model generations, instantiated here on a Qwen3-based Zen model (we use the 8B dense variant as the generator/student):

1. **Round 0**: Generate an initial batch of examples using the base Zen model. Train round-1 model.
2. **Round 1**: Use the improved round-1 model to generate a larger batch. Apply quality scoring. Train round-2 model.
3. **Round** n: Continue until quality score improvement $< 0.5\%$ per round.

The flywheel exploits the fact that a better model generates better data (higher quality scores, more diverse examples, fewer factual errors), which trains a yet-better model.

4.2 Collapse Prevention

Model collapse—where a model trained on its own outputs degrades—is prevented by:

- **Human data mixing**: Each training round mixes 30% human-curated data with 70% synthetic.

- **Distribution anchoring**: Synthetic data distribution is constrained to remain within KL-divergence ϵ of the real data distribution.
- **Diversity enforcement**: Deduplication at each round prevents the model from amplifying its own idiosyncratic patterns.

Table 4: Self-play flywheel: qualitative trend across rounds. Each round increases the data volume and the average quality score of the generated corpus, and downstream benchmark scores rise correspondingly before plateauing.

Round	Examples	Avg Quality	MT-Bench	MMLU
0 (base)	—	—	baseline	baseline
1	smaller	rising	↑	↑
2	larger	rising	↑↑	↑↑
3	larger	rising	↑↑↑	↑↑↑
4	largest	plateauing	↑↑↑	↑↑↑

Average generated-data quality and downstream MT-Bench/MMLU scores both improve monotonically across rounds, with the per-round gain shrinking as the flywheel approaches the stopping criterion (<0.5% quality improvement per round).

5 Domain-Specific Synthetic Data

5.1 Code Synthesis

For code training data, we employ a specialized code synthesis pipeline:

1. **Problem generation**: Generate algorithmic problems at specified difficulty levels (LeetCode Easy through Hard).
2. **Solution synthesis**: Generate solutions in 12 programming languages.
3. **Execution verification**: Run solutions against auto-generated test cases; reject non-passing solutions.
4. **Explanation generation**: Generate natural language explanations of the solution approach.

Execution-verified code data quality: 98.4% of accepted examples are fully correct (vs. 71.2% without verification).

5.2 Mathematical Reasoning

Mathematical data generation follows a structured approach:

- Generate problems by composing primitive operations with specified difficulty.
- Generate step-by-step solutions with explicit symbolic manipulation.
- Verify solutions using a computer algebra system (SymPy/Mathematica).
- Generate hints and worked examples for pedagogical utility.

Table 5: Domain-specific synthetic data volume and quality

Domain	Examples	Quality Score	Verification Rate
General instruction	2.1M	0.831	N/A
Code (Python/JS/etc.)	840K	0.894	98.4% (execution)
Mathematics	420K	0.912	97.8% (CAS verify)
Medical QA	280K	0.848	94.2% (citation)
Legal analysis	210K	0.824	91.8% (citation)
Financial analysis	320K	0.838	93.4% (source)
Multi-turn dialogue	630K	0.812	N/A
Total	**4.8M**	**0.847**	—

6 Diversity Analysis

6.1 Topic Coverage

Synthetic data topic coverage vs. web-scraped instruction data:

Table 6: Topic diversity comparison

Metric	Web-Scraped	Zen Synthetic (CSG)
Unique topics covered	1,840	2,384
Topic entropy (bits)	8.42	10.18
Tail topic coverage (<0.01%)	38%	71%
Average semantic distance	0.48	0.64

CSG data covers 30% more unique topics and has 21% higher entropy, indicating more uniform coverage of the knowledge space.

7 Human Evaluation

Blind human evaluation of 800 examples (400 CSG, 400 filtered web data) by domain expert raters:

Table 7: Human evaluation: CSG vs. filtered web data (1–5 scale)

Criterion	CSG	Web Data	Δ
Accuracy	4.38	3.91	+0.47
Helpfulness	4.28	3.94	+0.34
Clarity	4.24	3.84	+0.40
Depth	4.12	3.72	+0.40
Formatting	4.42	3.78	+0.64
Overall	**4.21**	**3.84**	**+0.37**

Human raters preferred CSG data in 71.4% of pairwise comparisons.

8 Downstream Task Improvements

Holding model and data volume fixed and swapping web-scraped instruction data for an equal volume of CSG data, we observe consistent downstream improvements across all evaluated benchmarks (MT-Bench, MMLU, HumanEval, MATH, GSM8K, IFEval). The largest relative gains appear on the instruction-following and code/reasoning benchmarks (IFEval, HumanEval, MATH), where the constitutional constraints and execution/CAS verification in the generation pipeline most directly improve data quality; gains on broad-knowledge MMLU are smaller. The direction and ordering of these improvements confirm the effectiveness of the CSG framework relative to web-data training.

9 Conclusion

The Zen Synthetic Data framework demonstrates that constitutional generation, automated quality scoring, and self-play data flywheels can produce training data that exceeds filtered web data on all measured quality dimensions. Models trained on the synthetic CSG corpus improve on MT-Bench and MMLU over web-data baselines, and human raters consistently prefer CSG data in pairwise comparisons. The self-play flywheel enables continuous quality improvement across training rounds without human annotation intervention. All generators and students in the pipeline are Qwen3-based Zen models (Apache-2.0).

References

[1] Taori, R. et al. Alpaca: A Strong, Replicable Instruction-Following Model. Stanford CRFM Blog, 2023.

[2] Wang, Y. et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions. *ACL*, 2023.

[3] Bai, Y. et al. Constitutional AI: Harmlessness from AI Feedback. *arXiv:2212.08073*, 2022.

[4] Shumailov, I. et al. The Curse of Recursion: Training on Generated Data Makes Models Forget. *Nature*, 2024.

[5] Mukherjee, S. et al. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. *arXiv:2306.02707*, 2023.